

KWEAVE

Knowledge Workflow: Embeddings, Automation, and Visual Exploration

Brent Thompson

Thompson Solutions Company

thompsonsolutionscompany@gmail.com

June 23, 2026

Abstract

Decades of curated scientific literature encode domain knowledge that is expensive to formalize: building an ontology and defining its relationships remains a slow, expert-driven process that goes well beyond named-entity recognition. We present KWEAVE (Knowledge Workflow: Embeddings, Automation, and Visual Exploration), an automated, multi-agent workflow that turns a curated literature corpus into a standards-compliant knowledge graph. An orchestrating large language model (LLM) coordinates specialist agents and reproducible tools that, in one ordered pipeline, discover a shared concept vocabulary from abstracts, mine each full-text paper for values and exact relational triples, and emit an OWL 2 ontology with JSON-LD instances interoperable with the Basic Formal Ontology (BFO), the Common Core Ontologies (CCO), and MDS-Onto. Relation extraction combines an LLM with the REBEL model so that causal and relational facts are recorded as stated; a LoRA fine-tuning step closes a learning loop on each finished ontology. Every run produces two deliverables—a GraphDB-ready repository and a cemento-compatible draw.io diagram—and is gated by structural validation (reasoner consistency, ontology-pitfall scanning, and upper-ontology alignment). We describe the system, its evolution across six pipeline generations into a single modular package, and an evaluation methodology grounded in reproducibility and explainability.

1 Introduction

General-purpose language models capture some of the connections latent in published research, but the majority of their learned representations are of uneven quality for any specific domain, and they do not by themselves yield an explicit, editable ontology. Knowledge graphs make such structure explicit, queryable, and reusable [7, 8]. The bottleneck that motivates this work is the long and labor-intensive process of *ontology creation and relationship definition*—the step beyond entity recognition and subject coverage that today requires trained ontologists and coordinated community effort [9].

We address this with KWEAVE, an *agentic* workflow: LLMs are given tools and run in a coordinated loop to produce a reproducible and explainable knowledge graph. Our contributions are:

1. A single ordered, multi-provider pipeline that converts a curated Zotero corpus into an OWL 2 ontology and JSON-LD instances, supporting both unsupervised (auto-discovered concepts) and supervised (provided concept list) modes.
2. A dual extraction stage in which an LLM mines per-concept values while REBEL extracts exact subject–predicate–object triples in parallel, capturing relational facts as stated.
3. Standards grounding to BFO/CCO/MDS-Onto with a mandatory validation gate (reasoner consistency, OOPS! pitfalls, and alignment coverage).

4. A LoRA learning loop that fine-tunes on each finished ontology’s terms so later runs improve on new material.
5. Two ready-to-use deliverables per run: a GraphDB import repository and a cemento draw.io diagram.

Suggested citations. Cite a general knowledge-graph reference (Hogan et al., *Knowledge Graphs*, ACM Computing Surveys 2021) and an LLM+KG roadmap (Pan et al., IEEE TKDE 2024) to frame the problem; cite the OBO Foundry (Smith et al., *Nature Biotechnology* 2007) for the cost of manual, coordinated ontology engineering.

2 Background and Related Work

Foundational ontologies. The Basic Formal Ontology (BFO) is a top-level ontology adopted as ISO/IEC 21838-2 [3]; the Common Core Ontologies extend it at the mid level, and MDS-Onto provides materials-data-science vocabulary. Grounding generated classes in these enables interoperability and reuse, following the design discipline established by the OBO Foundry [9].

LLM-based knowledge-graph construction. Recent surveys map a fast-moving space of LLM-driven extraction and fusion methods [4, 6, 8]. NeOn-GPT is the closest prior pipeline, translating natural-language descriptions into Turtle ontologies, and its reported limits—LLMs lacking the procedural and reasoning rigor for full ontology construction—directly motivate our tool-augmented, validated design [2]. Related extraction-from-text systems include schema-driven prompting approaches such as SPIRES/OntoGPT [10].

Relation extraction and agents. REBEL casts relation extraction as sequence generation, emitting triplets end-to-end [1]; we use it as a reproducible tool alongside the LLM. The multi-agent orchestration follows the conversational, tool-using paradigm of AutoGen [5], building on reasoning-and-acting and self-tooling methods such as ReAct [11] and Toolformer [12], with the LLM as planner and the neural models as callable components.

Suggested citations. Ontologies: add CCO and an MDS-Onto/SDLE reference for your group’s vocabulary. Ontology learning: Text2Onto (Cimiano & Völker, 2005) and OntoGPT/SPIRES (Caufield et al., *Bioinformatics* 2024). Agents/tool use: ReAct (Yao et al., 2023), Toolformer (Schick et al., 2023), and a recent multi-agent survey to complement AutoGen.

3 System Architecture

The workflow is a single linear pipeline with two hard ordering invariants: JSON-LD instances and the LoRA fine-tune run *only after* the ontology is built. The LLM acts as orchestrator and reasoner; the neural models (REBEL, the fine-tuned extractor) are versioned, seeded, logged tools. Table 1 summarizes the stages.

Provider-agnostic agents. Each agent is configured against any OpenAI-compatible endpoint, so the same code runs on hosted models or a local 7B-parameter Llama [13]. Structured outputs are obtained through typed result schemas (function/tool calling), and a fixed-choice taxonomy constrains the grounding tags—useful when smaller local models are employed.

Explainability and reproducibility. Model versions and seeds are pinned; each assertion carries provenance (source paper, tool, and confidence) modeled on the W3C PROV vocabulary [16]; REBEL triples are exported with a `prov:wasDerivedFrom` link to their source paper. The validation gate must pass before the repository is published.

Suggested citations. Structured output: cite the tool/function-calling mechanism you rely on (e.g. constrained/grammar-based decoding) and the PydanticAI/Instructor libraries. Provenance: PROV-O (W3C Recommendation, 2013). Local models: LLaMA (Touvron et al., 2023).

Step	Name	Function
0	Input	Fetch papers + PDF text from Zotero; choose supervised/unsupervised mode
1	Concepts	Extract concepts from abstracts; normalize to a shared 30–80 term list
2	Mine	Per paper: LLM extracts value+quote per concept; REBEL extracts triples
3	Consolidate	Reconcile concepts and triples into the final ontology terms
4	Ontology	Build OWL 2 TTL grounded in MDS/CCO/BFO; <i>validation gate</i>
5	JSON-LD	Emit per-paper and combined JSON-LD instances (GraphDB repo)
5b	Diagram	Tag concepts (study stage / supply chain); build cemento draw.io
6	LoRA	Fine-tune adapter on the final ontology terms (terminal step)

Table 1: The ordered KWEAVE pipeline. Steps 5, 5b, and 6 execute only after Step 4.

4 Implementation and Evolution

KWEAVE is implemented as a single modular Python package (**kw**) exposing one ordered runner. Papers and full text are retrieved through the Zotero API; the ontology and JSON-LD are produced by a Turtle/JSON-LD emitter that classifies concepts into MDS branches (measurement, material, process, device, characterization, reliability, sample, economics); output conforms to OWL 2 [17] and JSON-LD, and structural constraints can be expressed in SHACL [18]. The cemento diagram is generated with embedded MDS-Onto and cemento palette libraries.

The learning loop. After a domain ontology is finalized, a Low-Rank Adaptation (LoRA) [14] adapter is fine-tuned on the run’s final ontology terms and mined text→value pairs; quantized variants such as QLoRA [15] make this feasible on modest hardware. The intent is that successive runs reuse prior ontologies to ingest new material more accurately.

The current design is the consolidation of six pipeline generations (Table 2). Early versions explored classical NLP and transformer keyword extraction; mid versions introduced structured LLM extraction and two-stage discovery with full-text mining; later versions added provider independence and concept enrichment. The work described here unifies these—together with a previously separate ontology/JSON-LD emitter and a draw.io generator—into one deduplicated pipeline with a single data contract and a validation gate.

Gen.	Method	Key advance
V1	spaCy + TF-IDF	offline concept counts (baseline)
V2	KeyBERT	semantic keyphrases
V3	LLM structured	per-paper typed extraction
V4	Two-stage + full text	concept discovery + source quotes
V5	Provider-agnostic	any OpenAI-compatible model
V6	Enrichment	MDS tagging + concept-map diagrams
<i>This work</i> (KWEAVE): unified modular package; OWL 2/JSON-LD + diagram; REBEL + LoRA; validation.		

Table 2: Evolution of the pipeline into KWEAVE.

Suggested citations. Standards: OWL 2 (W3C, 2012), JSON-LD (W3C, 2020), SHACL (W3C, 2017). Tooling: ROBOT (Jackson et al., 2019) for reasoning/reporting; the GraphDB and Zotero documentation as software references. Fine-tuning: LoRA (Hu et al., 2021), QLoRA (Dettmers et al., 2023), and an instruction-tuning reference (e.g. Wei et al., FLAN, 2022).

5 Evaluation

Because manually constructing a reference ontology for every domain is impractical—and the cost and difficulty of manual ontology engineering is well established in the literature—we evaluate the *generated artifact* with structural metrics rather than a human study:

- **Logical consistency:** the OWL 2 ontology passes a DL reasoner (e.g. HermiT or ELK) with no unsatisfiable classes.
- **Modeling quality:** an ontology-pitfall scan (OOPS! [19]) reports no critical pitfalls.
- **Interoperability:** the fraction of classes that trace to a BFO/CCO/MDS parent (alignment ratio).

On a hand-built reference produced by an earlier run, all declared classes aligned to an upper parent, indicating the grounding step behaves as intended; a full multi-domain evaluation with a local 7B model is in progress. Reproducibility is supported by pinned model and seed values and by the provenance attached to every assertion.

Suggested citations. Ontology evaluation: OOPS! (Poveda-Villón et al., 2014) and a general ontology evaluation reference (e.g. Vrandečić, 2009; or OntoMetrics). Reasoners: HermiT (Glimm et al., 2014) and ELK (Kazakov et al., 2014). If you add a human-quality study later, cite an inter-annotator agreement methodology.

6 Discussion and Limitations

The architecture deliberately favors simplicity: a linear pipeline with one owner per step, no speculative agent loop. Open challenges include (i) the reliability of strict structured output from small local models, mitigated by fixed-choice schemas and conservative concept counts; (ii) entity resolution when merging REBEL triples with LLM concepts, so the same real-world entity becomes a single node; and (iii) the volume of supervision available for the LoRA loop, which at present seeds from existing ontologies and prior runs. Structural validation establishes internal soundness but not domain correctness, which still benefits from expert review.

Suggested citations. Entity resolution/linking: a neural entity-linking reference (e.g. Wu et al., BLINK, 2020). Reliability of LLM output: a survey on hallucination/factuality in LLMs. Small-model structured output: constrained-decoding references.

7 Conclusion and Future Work

We described KWEAVE, an automated agentic workflow that converts curated literature into a validated, standards-grounded knowledge graph and an editable diagram, with relational facts captured by REBEL and a per-ontology LoRA learning loop. Future work includes deeper entity resolution at the merge step, expanded structural and expert evaluation across domains, and using the trained LoRA adapter directly within the extraction agents to compound gains across runs.

References

- [1] Pere-Lluís Huguet Cabot and Roberto Navigli. REBEL: Relation Extraction By End-to-end Language generation. In *Findings of the ACL: EMNLP 2021*, pp. 2370–2381, 2021.
- [2] Nadeen Fathallah, Arunav Das, Stefano De Giorgis, Andrea Poltronieri, Peter Haase, and Liubov Kovriguina. NeOn-GPT: A Large Language Model-Powered Pipeline for Ontology Learning. In *ESWC 2024 Satellite Events*, pp. 36–50. Springer, 2024.
- [3] Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.

- [4] LLM-empowered Knowledge Graph Construction: A Survey. arXiv:2510.20345, 2025.
- [5] Qingyun Wu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155, 2023.
- [6] Large Language Models for Ontology Engineering: A Systematic Literature Review. *Semantic Web Journal*, 2025.
- [7] Aidan Hogan et al. Knowledge Graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.
- [8] Shirui Pan et al. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [9] Barry Smith et al. The OBO Foundry: Coordinated Evolution of Ontologies to Support Biomedical Data Integration. *Nature Biotechnology*, 25:1251–1255, 2007.
- [10] J. Harry Caufield et al. Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES). *Bioinformatics*, 40(3), 2024.
- [11] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*, 2023.
- [12] Timo Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023.
- [13] Hugo Touvron et al. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971, 2023.
- [14] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022 (arXiv:2106.09685).
- [15] Tim Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*, 2023.
- [16] World Wide Web Consortium. PROV-O: The PROV Ontology. W3C Recommendation, 2013.
- [17] W3C OWL Working Group. OWL 2 Web Ontology Language Document Overview (2nd Edition). W3C Recommendation, 2012.
- [18] Holger Knublauch and Dimitris Kontokostas. Shapes Constraint Language (SHACL). W3C Recommendation, 2017.
- [19] María Poveda-Villón, Asunción Gómez-Pérez, and Mari Carmen Suárez-Figueroa. OOPS! (Ontology Pitfall Scanner!). *Int. J. Semantic Web Inf. Syst.*, 10(2):7–34, 2014.